

Functions

- A function is a reusable block of code that performs a specific task
- A function is identified by its name (for example, `sqrt`)
- Parentheses () indicate that the function is being called
- Values inside the parentheses are called arguments, which are passed to the function as input
- The function executes its internal code using the given arguments
- The function computes a result, which is returned to the caller
- The returned value can be printed, stored in a variable, or used in an expression
- The core of the function code block should be indented by 4 spaces.

wrong code

```
def sum(a,b):  
return a+b
```

correct code

```
def sum(a,b):  
    return a+b
```

Reference: <https://docs.python.org/3/tutorial/controlflow.html#defining-functions>

Function Call and Return



```
In [ ... def sum(x, y):  
        # print(x)  
        print(x+y)  
        # return x+y
```

```
sum(1,2)
# print(a)
```

3

```
In [... def foo(a):
        sum = 0
        for i in a:
            sum = sum + i
        return sum/len(a)
foo([2,3,4])
```

Out[... 3.0

```
In [... def addElements(a, b):
        return a + b

addElements(2, 3)
addElements(2, 3.1)
addElements(2.3, 4.5)
addElements("python", " workshop")
# addElements(2) # error!
```

Out[... 5

Out[... 5.1

Out[... 6.8

Out[... 'python workshop'

```
In [... # when the order is important!
```

```
def subtract(a,b):
    return a-b
```

```
subtract(2,3)
subtract(3,2)
subtract(b=3,a=2)
```

Out[... -1

Out[... 1

Out[... -1

```
In [... def greet(name, first='Hello,'):  
# def greet(name, first):  
    """  
    docstring: write something useful!  
    """  
    # print(f'{first} {name}!')  
    return f'{first} {name}!'
```



```
print(greet('Sindhu', 'Hello,'))
```

Hello, Sindhu!

```
In [... import numpy as np  
np.arange(0,10,1)
```

```
Out[... array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
In [... def multdiv(x, y, mult=True):  
    """  
    this is a horrible function, you shouldn't use th  
    """  
    if mult:  
        result = x * y  
    else:  
        result = x / y  
    return result
```



```
multdiv(2, 2)  
multdiv(2, 2, True)  
multdiv(2, 2, False)
```

```
Out[... 4
```

```
Out[... 4
```

```
Out[... 1.0
```

```
In [... def repeat(s, exclaim=False):  
    "A completely useless function, but hey it has a  
    result = s * 3  
    if exclaim:  
        result += "!!!"  
    return result
```



```
repeat("Yay, ")
```

```
# repeat(3)

# repeat('Yay, ', False)
repeat('Woo ', True)
```

```
Out[... 'Yay, Yay, Yay, '
```

```
Out[... 'Woo Woo Woo !!!'
```

```
In [... # default values for parameters
def addElements(a, b=3):
    return a + b
```

```
addElements(2)
addElements(2, 5)
```

```
Out[... 5
```

```
Out[... 7
```

A complete example

with some advanced bits..

```
In [... def foo(a, b=4):
        c = 3
        return 3+c+a
foo(4,0)
# print(c)
```

```
Out[... 10
```

```
In [... # --- GLOBAL SCOPE ---
x = "Global Variable"

def example_function(required_param, default_param="D
    """
    DOCSTRING: This is where you explain what the fun
    It takes two parameters and returns a formatted s
    """
```

```

# --- LOCAL SCOPE ---
# Variables defined here only exist inside this f
internal_val = "Local Variable"

result = f"{required_param} | {default_param} | {"
return result
# print(internal_val)

```

```

In [... # 1. Calling with Positional Arguments
print(example_function("Hello", "Custom Value"))

```

Hello | Custom Value | Local Variable

```

In [... # 2. Calling with Keyword Arguments (Order doesn't ma
print(example_function(default_param="Keyword", requi

```

World | Keyword | Local Variable

```

In [... # 3. Using Default Argument
print(example_function("Just Required"))

```

Just Required | Default Value | Local Variable

```

In [... # 4. Scope demonstration
print(x) # Works (Global)
# print(internal_val) # NameError (Local scope only)

```

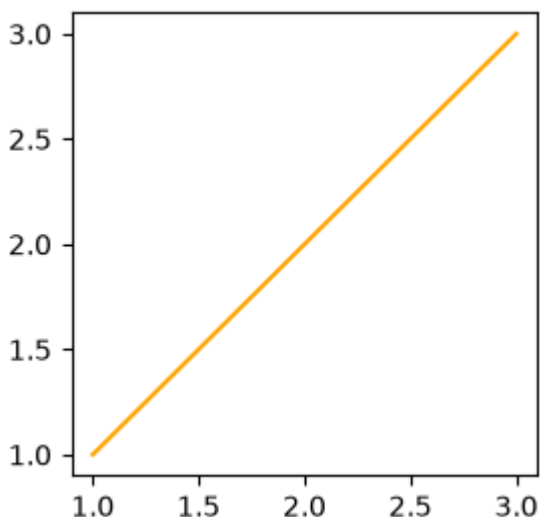
Global Variable

```

In [... import matplotlib.pyplot as plt
fig,ax= plt.subplots(figsize=(3,3))
ax.plot([1,2,3],[1,2,3],color='orange')

```

Out[... [<matplotlib.lines.Line2D at 0x12ffa1e20>]



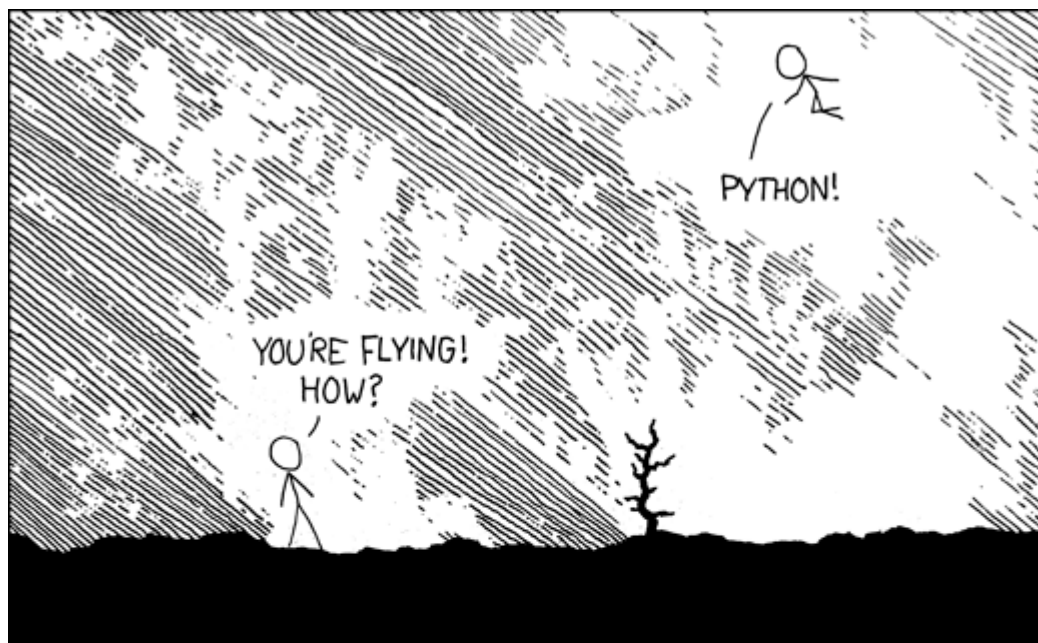
```
In [... # 5. Accessing Docstring
print(example_function.__doc__)
```

DOCSTRING: This is where you explain what the function does. "

It takes two parameters and returns a formatted string.

Modules and Packages and `__main__`

Python files are modules. When run directly, **name** equals **'main'**. See `example_import.py` and `example_main.py`. For more details check [Python's documentation](#).



Source: XKCD

In the following code block:

- `import math` loads the standard Python math module, which provides mathematical functions and constants
- `math.sqrt(16)` calls the `sqrt` (square root) function from the module with input value `16` ; the result is `4.0`
- The result is a float because `math.sqrt` always returns a floating-point value
- Without `import math` , `sqrt` is not available

Reference: <https://docs.python.org/3/library/math.html>

```
In [... import math

math.sqrt(3.498572394)
```

```
Out[...] 1.8704471107197873
```

Ways to use functions/constants from a module

```
In [... # import a single function
from random import sample
sample("Luke", 2)
```

```
Out[...] ['L', 'u']
```

```
In [... # import without an alias. here you need to mention t
import numpy
print(numpy.absolute(-2))
print(numpy.e)
```

```
2
2.718281828459045
```

```
In [... # import (with an alias). most common use. the aliase

import numpy as np
print(np.absolute(-2))
```

```
2
```

```
In [... # import the whole module without any alias. this all
```

```
# from numpy import *
import math
import numpy as np
print(math.e)
print(np.e==math.e)
```

2.718281828459045

True

In [... *# more on namespace <https://realpython.com/python-namespaces-for-beginners/>*

```
x = 3

def square(y):
    """
    this function squares a variable
    """
    y = 2
    return y**2

# y # will return an error

import sum_module
sum_module.sum(2,3)
```

Out[... 5

In [... *# why should we use loops and functions?*

```
10**2+2
20**2+2
30**2+2
```

Out[... 102

Out[... 402

Out[... 902

In [...

```
lst = [10,20,30]

# lst= list(range(10000))

def func(x):
    return x**2+2

for i in lst:
```



```
print(func(i))
```

102

402

902

```
In [... # function use-case
import matplotlib.pyplot as plt
def plot_wave(ax, frequency):
    x = np.linspace(0, 2*np.pi, 100)
    y = np.sin(frequency * x)
    ax.plot(x, y)
    ax.set_title(f"$f = {frequency}$")

fig, axes = plt.subplots(1, 3, figsize=(6, 2), sharey

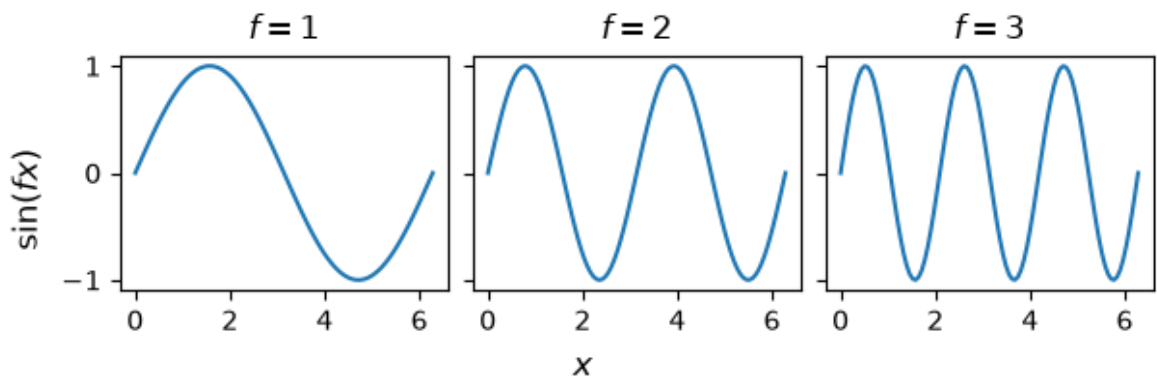
fig.supylabel(r"$\sin (fx)$")
fig.supxlabel(r'$x$')

for i in range(0,3):
    plot_wave(axes[i], i+1)

# for i, elem in enumerate(axes):
#     plot_wave(elem, i+1)
```

Out[... Text(0.02, 0.5, '\$\sin (fx)\$')

Out[... Text(0.5, 0.01, '\$x\$')



```
In [... lst1 = ('n1', 'n2')
```

```
a, b = lst1
```

```
a
```

```
b
```

Out[... 'n1'

Out [... 'n2'